Laboratório de Concorrência · Aula 10

Laís Lima Sampaio · RA 23288786-2 · Turma C
Gerado em 09/10/2026, 21:40:42 · tempo registrado no cronômetro: 75:02

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação é quando uma atividade depende de outra para ser concluída, por exemplo, para se fazer um relatório de uma pesquisa deve-se primeiro fazer a pesquisa. E competição é quando tarefas precisam de um mesmo recurso, para ser concluídas, por exemplo, quando 2 pessoas precisam de um mesmo carro, uma para ir ao mercado e outra para ir à farmácia.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

O esperado seria 2.000.000 mas a impressão imagino ser imprevisível, já que quem controla é compilador e pode acontecer uma condição de corrida, duas threads podem ler o mesmo valor e somar 1 à um valor antigo, que outra threads já alterou.

1ª execução (n = 1.000.000)

2000000

2ª execução (n = 1.000.000)

1094299

3ª execução (n = 1.000.000)

1358876

Saída do programa

```
esperado: 2000000
obtido:   1358876
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram com o esperado e não foram iguais nas três execuções. Pois as threads no processador competem pelo mesmo recurso que no caso é a variável. E uma das threads pode ter lido um valor ao mesmo tempo e ao invés de somar por cima do valor atualizado ela foi somando sobre o valor antigo.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Thread t1 -> lê 1
Thread t2 -> lê 1
Thread t1 -> soma 1 que gera 2
Thread t2 -> soma 1 que gera 2 também
Thread t1 -> grava 2
Thread t2 -> grava 2

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com 1000 o esperado foi igual ao obtido. Se torna perigoso em produção pois se for um sistema por exemplo de um aeroporto uma pessoas poderia perder o voo por pegar o mesmo assento de outra pessoa.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

ABABAB → 6
AABBBA → 4
AAABBB → 8
BBAAAB → 6
BBBAAA → 7
ABBBAA → 4

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
```

```
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

20 ordens de combinações possíveis. Apenas 1.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Quando vc faz ABBBAA. A operacão A sobreescreveu o resultado da operação de B.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acho que a synchronized é a mais rápida e acho que nenhuma será incorreta.

Saída do programa

```
1.2s
esperado:          2000000
sem sincronizacao: 1014781 em 2 ms
synchronized:      2000000 em 52 ms
AtomicInteger:     2000000 em 61 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

AtomicInteger e synchronized. Porque são feitas mais verificações.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolveria, porque apesar de isolar aquela operação você não garante a ordem das operações. Seria preciso colocar as operações em um mesma operação.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não pois o programa nunca executará o código já que cada thread espera um recurso que está com a outra e vice versa.

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas

A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

'A' tinha X e queria Y e 'B' tinha Y e queria X ficaram em impasse.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Porque o item B espera o A terminar de usar para pegar o recurso. Em um setor que usa a mesma impressora.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

200 ms se for ao mesmo tempo. Com 1 trabalhador 600 ms. Com 6 trabalhadores 100 ms

Tempo com 1 trabalhador

600

Tempo com 3 trabalhadores

200

Tempo com 6 trabalhadores

100

Saída do programa

```
354 ms
trabalhador 5 fez a tarefa 6
trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
```

```
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Porque eles passam mensagens não se comunicam por memória.

Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
        /box/main.go:9 +0x38
exit status 2 / 5
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro trava porque canais sem buffer exigem um emissor e um receptor ativos ao mesmo tempo, enquanto o canal com buffer salva o dado na memória e libera a linha para avançar.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O rust não permite a thread mudar a variável pois ela pertence a funcão main.

Saídas (o erro e o resultado com move)

```
Exit 1
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is owned by the current
function
 ⟶ HelloWorld.rs:7:27
  |
7 |     let t = thread::spawn(|| {
  |                           ^^ may outlive borrowed value `contador`
8 |         contador += 1;                       // a thread altera a variável de main
  |         -------- `contador` is borrowed here
```

```
   |
note: function requires argument type to outlive `'static`
 ⟶ HelloWorld.rs:7:13
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O move faz a thread alterar uma cópia independente do número básico, deixando o contador original da main intacto em 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java o erro surge na execução e em Rust na compilação, trazendo a vantagem de software livre de bugs de concorrência ao custo de uma curva de aprendizado mais complexa para satisfazer o compilador.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Pois o código possui uma única thread.

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem foi A, D, C, B porque D é síncrono (imediato), C está na fila de promessas (prioridade máxima) e B vai para a fila de timers, que só roda após o código principal e as promessas terminarem.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer atrasou porque a função ocupado(300) bloqueou a única thread do Node.js por 300 ms, o que significa que um cálculo pesado em

produção trava o servidor inteiro e faz todos os outros usuários esperarem na fila.

Fechamento: Quem impede a corrida?

Java sem sincronização

O programador, com sincronização

Java com synchronized

O compilador

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O programador, com sincronização

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

O DeadLock. Os semáforos

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

A forma como nos conduz a pensar.